ক্লাস ৬

Dart Object-Oriented Programming

Complete Study Notes & Practical Workbook

Class, Object, Constructor, Named Constructor,
Properties, Methods & Core OOP Concepts

DART / FLUTTER • BEGINNER TO INTERMEDIATE

Language format

Explanations, code examples, and terminology are in English. Class label, practice questions, and homework are in Bangla.

Learning goal: Design reusable Dart classes, initialize objects safely, work with methods and properties, and apply OOP ideas in Flutter.

Contents & Learning Plan

Work through the topics in order. After each topic, run the code, answer the questions, and complete the homework without copying the example directly.

01 OOP Fundamentals

02 Class

03 Object

04 Constructor

05 Named Constructor

06 Properties / Fields

07 Methods

08 Encapsulation and Abstraction

09 Inheritance and Polymorphism

10 Flutter Connection and Mini Project

Before you begin

Install Dart (included with Flutter), open a Dart project in VS Code, and run sample code using `dart run`. These examples use Dart without third-party packages.

Suggested lesson plan

Part A: Fundamentals, classes, objects, constructors, and named constructors. **Part B:** Properties, methods, OOP principles, and a mini project.

Good habits

Use PascalCase for class names, lowerCamelCase for fields and methods, explicit types where they improve readability, and `final` for values that should not be reassigned.

TOPIC 01 / 10

OOP Fundamentals

Object-Oriented Programming (OOP) organizes software around objects that combine state (data) and behavior (actions). Dart is object-oriented: almost everything you use is an object. OOP helps structure Flutter applications, avoid duplicated logic, and make code easier to test and maintain.

Example (Dart)

```
void main() {
  final String appName = 'Study Planner';
  print(appName);
}
```

Key Concepts

Encapsulation: Keep related data and behavior together and limit direct access to internal details.

Abstraction: Expose only the operations a consumer needs.

Inheritance: Reuse behavior through a parent-child relationship.

Polymorphism: Use a shared interface while allowing different implementations.

অনুশীলনী প্রশ্ন

1. OOP কী? বাস্তব জীবনের দুটি উদাহরণ দিয়ে ব্যাখ্যা করো।
2. Encapsulation ও Abstraction-এর পার্থক্য কী?
3. Flutter অ্যাপ তৈরিতে OOP কেন উপকারী?

বাড়ির কাজ (Homework)

1. একটি Library Management App-এর জন্য অন্তত পাঁচটি সম্ভাব্য object-এর নাম লিখো। প্রতিটির data এবং behavior আলাদা করে দেখাও।
2. OOP-এর চারটি মূল নীতি বাংলায় নিজের ভাষায় ব্যাখ্যা করো।

TOPIC 02 / 10

Class

A class is a blueprint for creating objects. It defines which fields (properties) and methods an object has. The class itself is not an individual real-world item; each instance created from it is an object.

Example (Dart)

```
class Student {
  String name = 'Unknown';
  int grade = 6;

  void introduce() {
    print('My name is $name. Grade: $grade');
  }
}

void main() {
  final student = Student();
  student.name = 'Nadia';
  student.introduce();
}
```

Key Concepts

Class declaration: Use the class keyword and a PascalCase name, e.g., StudentProfile.

Instantiation: Student() creates a new Student object.

Member access: Use the dot operator to read properties or call methods.

অনুশীলনী প্রশ্ন

1. Class এবং Object-এর মধ্যে পার্থক্য কী?
2. `Student()` কী কাজ করে?
3. Class-এর নাম লেখার ক্ষেত্রে কোন naming convention ব্যবহার করা উচিত?

বাড়ির কাজ (Homework)

1. `Book` নামে একটি class তৈরি করো। এর title, author এবং pages properties রাখো। দুইটি object তৈরি করে তাদের তথ্য print করো।
2. `Car` class লিখে brand এবং model properties তৈরি করো।

TOPIC 03 / 10

Object

An object is an instance of a class. Multiple objects can share the same blueprint while holding different values. Changes to one instance do not automatically change another instance.

Example (Dart)

```
class Product {
  String name = '';
  double price = 0;
}

void main() {
  final p1 = Product();
  p1.name = 'Keyboard';
  p1.price = 1200;

  final p2 = Product();
  p2.name = 'Mouse';
  p2.price = 650;

  print('${p1.name}: ${p1.price}');
  print('${p2.name}: ${p2.price}');
}
```

Key Concepts

Identity: p1 and p2 refer to distinct objects.

State: Each object has its own property values.

Behavior: Each object can call methods defined in its class.

অনুশীলনী প্রশ্ন

1. একই class থেকে তৈরি দুটি object-এর value আলাদা হতে পারে কেন?
2. Object-এর state বলতে কী বোঝায়?

বাড়ির কাজ (Homework)

1. `Mobile` class থেকে তিনটি আলাদা object তৈরি করো। প্রত্যেকটির brand ও price ভিন্ন রাখো।
2. একটি object-এর property পরিবর্তন করে আগের ও পরের value print করো।

TOPIC 04 / 10

Constructor

A constructor initializes an object when it is created. It has the same name as its class and no return type. Constructor parameters help require values at creation time. Dart supports concise initializing formals using this.field.

Example (Dart)

```
class Student {
  final String name;
  final int age;

  Student(this.name, this.age);

  void showInfo() => print('$name is $age years old');
}

void main() {
  final a = Student('Rafi', 17);
  a.showInfo();
}
```

Key Concepts

Default constructor: If you declare no constructor, Dart supplies an unnamed no-argument constructor where applicable.

Generative constructor: Creates a new instance of its class.

Initializing formal: this.name assigns a constructor argument to the corresponding field.

Required values: final fields should be initialized before the constructor completes.

অনুশীলনী প্রশ্ন

1. Constructor কী এবং কখন execute হয়?
2. `this.name` ব্যবহার করার সুবিধা কী?
3. Constructor এবং method-এর পার্থক্য কী?

বাড়ির কাজ (Homework)

1. `Employee` class তৈরি করো যেখানে constructor দিয়ে name, id এবং salary সেট হবে।
2. `Rectangle` class-এর constructor দিয়ে width ও height সেট করে area print করো।

TOPIC 05 / 10

Named Constructor

Named constructors provide multiple clearly labeled ways to create instances of the same class. They are written as ClassName.constructorName(...). They are useful when data arrives in different formats or when you need sensible defaults.

Example (Dart)

```
class UserProfile {
  final String name;
  final String role;

  UserProfile(this.name, this.role);

  UserProfile.guest()
      : name = 'Guest',
        role = 'viewer';

  UserProfile.admin(String username)
      : name = username,
        role = 'admin';
}

void main() {
  final guest = UserProfile.guest();
  final admin = UserProfile.admin('Ayesha');
  print('${guest.name} / ${guest.role}');
  print('${admin.name} / ${admin.role}');
}
```

Key Concepts

Syntax: ClassName.named(...) declares and invokes a named constructor.

Initializer list: The colon syntax initializes fields before the body runs.

Design use: Provide distinct, easy-to-understand construction paths.

অনুশীলনী প্রশ্ন

1. Named constructor কেন ব্যবহার করা হয়?
2. Normal constructor ও named constructor-এর syntax-এ কী পার্থক্য?
3. Initializer list কী কাজে লাগে?

বাড়ির কাজ (Homework)

1. `Account` class-এ `Account.guest()` এবং `Account.premium(String name)` named constructor তৈরি করো।

2. `Temperature` class-এ Celsius ও Fahrenheit থেকে object তৈরির জন্য দুটি named constructor লিখো।

TOPIC 06 / 10

Properties / Fields

Properties (fields) store the state of an object. Dart provides instance fields, static fields, getters, and setters. Prefer final where the value should not change after initialization. A leading underscore makes an identifier library-private in Dart, not class-private.

Example (Dart)

```
class BankAccount {
  final String owner;
  double _balance;
  static const String currency = 'BDT';

  BankAccount(this.owner, this._balance);

  double get balance => _balance;

  void deposit(double amount) {
    if (amount <= 0) {
      throw ArgumentError('Amount must be positive');
    }
    _balance += amount;
  }
}

void main() {
  final account = BankAccount('Sami', 500);
  account.deposit(200);
  print('${account.balance} ${BankAccount.currency}');
}
```

Key Concepts

Instance field: Each object has its own value.

Static field: Belongs to the class rather than a specific instance.

Getter: Provides controlled read access, e.g., account.balance.

Privacy: _balance is private to its Dart library.

final vs const: final is assigned once at runtime; const is a compile-time constant.

অনুশীলনী প্রশ্ন

1. Property বা field কী?
2. `final` ও `const`-এর পার্থক্য কী?
3. Getter ব্যবহারের সুবিধা কী?

4. Dart-এ `_balance` কীভাবে private হয়?

বাড়ির কাজ (Homework)

1. `Student` class-এ name, roll এবং একটি private `_marks` field তৈরি করো। Getter দিয়ে marks দেখাও।
2. `Product` class-এ discountPercentage অনুযায়ী finalPrice getter তৈরি করো।

TOPIC 07 / 10

Methods

Methods are functions defined inside a class. They represent behaviors and can read or change the object state. Methods can have positional parameters, named parameters, return types, and can be asynchronous.

Example (Dart)

```
class Calculator {
  int add(int a, int b) => a + b;

  double divide(double a, double b) {
    if (b == 0) {
      throw ArgumentError('Cannot divide by zero');
    }
    return a / b;
  }

  void greet({required String name}) {
    print('Hello, $name');
  }
}

void main() {
  final calc = Calculator();
  print(calc.add(4, 6));
  print(calc.divide(10, 2));
  calc.greet(name: 'Mira');
}
```

Key Concepts

void method: Performs work but returns no useful value.

Returning method: Declares a return type such as int or double.

Named parameters: Use curly braces in declaration and labels at call site.

Validation: Reject invalid input rather than producing misleading results.

অনুশীলনী প্রশ্ন

1. Method এবং normal function-এর পার্থক্য কী?
2. `void` কখন ব্যবহার করা হয়?
3. Named parameter-এর একটি সুবিধা লেখো।

বাড়ির কাজ (Homework)

1. `Calculator` class-এ subtract, multiply এবং modulus method যোগ করো।
2. `ShoppingCart` class তৈরি করো; addItem ও totalPrice method লিখো।

TOPIC 08 / 10

Encapsulation and Abstraction

Encapsulation groups state and behavior into an object while controlling access to internal state. Abstraction hides implementation details behind a simple interface. In Dart, private names begin with underscore and are private to the library.

Example (Dart)

```
abstract class PaymentMethod {
  void pay(double amount);
}

class CardPayment implements PaymentMethod {
  @override
  void pay(double amount) {
    print('Paid $amount using a card');
  }
}

void main() {
  PaymentMethod method = CardPayment();
  method.pay(250);
}
```

Key Concepts

Interface: PaymentMethod describes what a payment type must do.

Implementation: CardPayment supplies the concrete behavior.

Abstraction boundary: Calling code knows pay() but not how card processing works.

অনুশীলনী প্রশ্ন

1. Encapsulation-এর বাস্তব উদাহরণ দাও।
2. Abstract class সরাসরি instantiate করা যায় কি?
3. Interface ও implementation-এর কাজ কী?

বাড়ির কাজ (Homework)

1. `Shape` abstract class-এ `area()` method ঘোষণা করো এবং `Circle`-এ implement করো।
2. `Wallet` class-এ private balance ও deposit/withdraw method তৈরি করো।

TOPIC 09 / 10

Inheritance and Polymorphism

Inheritance lets a subclass extend behavior from a superclass using extends. Polymorphism allows different subclasses to respond to the same method call in their own way. Prefer composition over inheritance when the relationship is not genuinely "is-a".

Example (Dart)

```
class Animal {
  void speak() => print('Some animal sound');
}

class Dog extends Animal {
  @override
  void speak() => print('Woof');
}

class Cat extends Animal {
  @override
  void speak() => print('Meow');
}

void main() {
  final List<Animal> animals = [Dog(), Cat()];
  for (final animal in animals) {
    animal.speak();
  }
}
```

Key Concepts

extends: Dog and Cat inherit from Animal.

@override: Marks a subclass implementation of an inherited member.

Dynamic dispatch: The actual object determines which speak() method executes.

super: Can access superclass behavior from a subclass.

অনুশীলনী প্রশ্ন

1. Inheritance কীভাবে code reuse করতে সাহায্য করে?
2. Polymorphism-এর উদাহরণ কী?
3. `@override` কেন ব্যবহার করা হয়?

বাড়ির কাজ (Homework)

1. `Vehicle` parent class এবং `Bike` ও `Bus` child class বানিয়ে `move()` override করো।

2. `Employee` parent ও `Manager` child class দিয়ে একটি salary display system তৈরি করো।

TOPIC 10 / 10

Flutter Connection and Mini Project

Flutter widgets are Dart classes. A StatelessWidget overrides build(), while a StatefulWidget holds mutable UI state in a separate State object. An OOP model can represent data independent of the UI. The following example creates a reusable Task model.

Example (Dart)

```
class Task {
  final String title;
  bool isDone;

  Task({required this.title, this.isDone = false});

  void complete() {
    isDone = true;
  }

  String get status => isDone ? 'Done' : 'Pending';
}

void main() {
  final task = Task(title: 'Learn Dart OOP');
  print('${task.title}: ${task.status}');
  task.complete();
  print('${task.title}: ${task.status}');
}
```

Key Concepts

Model class: Represents app data and its basic behavior.

Constructor: Uses required named argument title and default isDone.

Method and getter: complete() changes state; status derives a readable value.

Flutter use: Render Task values in a ListView or other widget.

অনুশীলনী প্রশ্ন

1. Flutter Widget-এর সঙ্গে class-এর সম্পর্ক কী?
2. Model class ব্যবহার করলে কী সুবিধা হয়?
3. `status` getter-এর value কখন পরিবর্তন হবে?

বাড়ির কাজ (Homework)

1. Task model ব্যবহার করে কমপক্ষে পাঁচটি Task তৈরি করো এবং প্রতিটির status print করো।

2. Bonus: Flutter-এ একটি Task list screen বানাও যেখানে button চাপলে Task complete হবে।

ক্লাস ৬ - চূড়ান্ত মূল্যায়ন

Revision and Final Assignment

Build a small console-based student management system using classes and objects. Use constructors, named constructors, fields, getters, and methods. Keep data valid and avoid unnecessary global variables.

Expected design

```
class Student {
  final String name;
  final int roll;
  int _marks;

  Student(this.name, this.roll, this._marks);

  Student.newAdmission(String name, int roll)
      : this(name, roll, 0);

  int get marks => _marks;

  void updateMarks(int value) {
    if (value < 0 || value > 100) {
      throw ArgumentError('Invalid marks');
    }
    _marks = value;
  }

  bool get isPassed => _marks >= 33;
}
```

চূড়ান্ত প্রশ্ন

1. Class, Object, Constructor এবং Method - প্রত্যেকটির কাজ সংক্ষেপে লেখো।
2. Named constructor কখন ব্যবহার করবে? একটি উদাহরণ দাও।
3. Encapsulation, Inheritance ও Polymorphism-এর পার্থক্য ব্যাখ্যা করো।

চূড়ান্ত হোমওয়ার্ক

1. উপরের Student class ব্যবহার করে পাঁচজন শিক্ষার্থীর object তৈরি করো।
2. একজন নতুন শিক্ষার্থীকে named constructor দিয়ে তৈরি করো।
3. Marks update করে pass/fail output দেখাও।
4. Bonus: Flutter-এর ListView-তে শিক্ষার্থীদের নাম ও result দেখাও।

Self-check checklist

☐ I can create a class and objects. ☐ I can use constructor parameters. ☐ I can write named constructors.

☐ I understand fields and getters. ☐ I can write and call methods. ☐ I can explain OOP principles.

End of Class 6 • Dart OOP Learning Workbook